VEILLE TECHNOLOGIQUE · LIVRABLE 1

Introduction aux API et à l'architecture REST

Avec FastAPI (Python) — de la conception au déploiement

Groupe : Groupe 1

Module : Création et déploiement d'API

Document : Synthèse détaillée — 13 chapitres

Date : Octobre 2026

FastAPI · HTTP · REST · OpenAPI · OAuth2 / JWT · Docker

p1ia_groupe-1

0 Avant-propos et objectifs

Ce document constitue le livrable de synthèse du travail de veille. Il reprend chaque concept du plan, en explique le « pourquoi », le « comment » et l'illustre par un exemple concret, avec un fil rouge : construire et déployer une API REST avec FastAPI.

Fiche du livrable

Nom du dossier	`p1ia_groupe-1_creation-deploiement-api`
Livrables	1) ce document PDF de synthèse — 2) `diapositives.pptx` (pourquoi / comment / exemple) — 3) quatre cheatSheets PDF
Public	Apprenants en développement web / API, niveau débutant à intermédiaire
Prérequis	Bases de Python, notion de HTTP, VSCode et Postman installés
Outils	Python 3.11+, FastAPI, Uvicorn, Pydantic, Docker, Postman, Git

Objectifs pédagogiques

- **Comprendre** ce qu'est une API, son rôle et les différents styles (REST, SOAP, GraphQL, gRPC).
- **Distinguer** les architectures (client-serveur, monolithe, SOA, microservices) et savoir les comparer.
- **Maîtriser** HTTP : requêtes, réponses, méthodes, codes de statut, formats d'échange.
- **Concevoir** une API REST cohérente : ressources, URIs, versioning, pagination, erreurs.
- **Développer** une API avec FastAPI : validation Pydantic, dépendances, documentation automatique.
- **Sécuriser** l'API : authentification, autorisation, OAuth2, JWT, OWASP API Top 10.
- **Déployer** l'API sur une plateforme open source (Coolify, Dokku, CapRover) avec Docker.

> **POURQUOI CE DOCUMENT EST-IL PLUS DÉTAILLÉ ?**
> Le cahier des charges demande un document de synthèse « plus détaillé (avec du texte) que les autres livrables ». Les diapositives servent à présenter, les cheatSheets à réviser : ce PDF sert de référence écrite, avec le raisonnement complet, les tableaux comparatifs et les exemples de code.

Fil rouge : l'API « Tâches »

Tout au long du document, nous utilisons un exemple simple et réaliste : une API de gestion de tâches (*todo list*). Elle expose des ressources `/tasks`, permet de créer, lire, modifier et supprimer des tâches, et sera sécurisée puis déployée. Ce fil rouge permet de relier la théorie (HTTP, REST) à la pratique (FastAPI, Docker).

Sommaire

I. Introduction générale aux API
Définition · rôle · domaines · avantages et limites

II. Classification des API
Styles web · politiques de publication · critères de choix

III. Architecture des applications et des API
Client-serveur · monolithe · SOA · microservices

IV. Fonctionnement d'une API Web
Communication · cycle d'appel · requête · réponse · formats

V. Protocole HTTP et méthodes
HTTP/1.1 → HTTP/3 · GET, POST, PUT, PATCH, DELETE

VI. Codes de statut HTTP
1xx à 5xx · gestion des erreurs · RFC 9457

VII. Architecture REST
Contraintes · ressources · HATEOAS · bonnes pratiques

VIII. Documentation des API
OpenAPI · Swagger UI · ReDoc · bonnes pratiques

IX. Introduction à FastAPI
Installation · routes · Pydantic · dépendances · tests

X. Programmation synchrone et asynchrone
Event loop · async/await · pièges · choix

XI. Sécurité : authentification et autorisation
OWASP · OAuth2 · JWT · bonnes pratiques

XII. Plateformes de déploiement open source
Coolify · Dokku · CapRover · Docker

XIII. Références
Documentation officielle, RFC, normes, outils

Chaque chapitre suit la même logique : **définition → pourquoi → comment → exemple**, afin de faciliter la lecture, la révision et la présentation orale.

I Introduction générale aux API

Une API est un contrat qui permet à deux logiciels de communiquer sans connaître leurs implémentations respectives. C'est la brique de base de l'informatique moderne connectée.

1.1. Définition d'une API

Une **API** (*Application Programming Interface*, interface de programmation applicative) est un ensemble de règles, de types de données et de points d'entrée qui permet à un programme d'utiliser les fonctionnalités d'un autre programme. Elle définit *ce que* l'on peut demander et *comment* le demander, mais masque *comment* c'est réalisé à l'intérieur.

On peut la comparer au **menu d'un restaurant** : le menu décrit les plats disponibles et la manière de les commander ; il ne décrit ni la cuisine, ni le fournisseur, ni les recettes. Le client (l'application appelante) n'a pas besoin de connaître la cuisine (l'implémentation) pour être servi.

API locale / bibliothèque

Une classe ou un module que l'on importe dans son propre code (ex. `json.dumps()`, une bibliothèque de calcul). L'appel se fait en mémoire, dans le même processus.

API Web / distante

Une interface exposée sur le réseau via HTTP (ex. `GET https://api.exemple.com/tasks`). L'appel traverse le réseau et suit un contrat documenté (OpenAPI, par exemple).

Les éléments constitutifs d'une API Web

Élément	Rôle	Exemple
Endpoint	Adresse d'une opération exposée	`GET /api/v1/tasks`
Contrat	Formats d'entrée/sortie, erreurs, règles	Schéma OpenAPI 3.1
Requête	Ce que le client envoie	Méthode, en-têtes, corps JSON
Réponse	Ce que le serveur renvoie	Code 200, JSON, en-têtes
Authentification	Qui appelle l'API	Clé API, JWT, OAuth2
Quotas / limites	Protection et équité d'usage	1 000 requêtes / heure

EXEMPLE CONCRET

L'application mobile de la SNCF appelle une API pour obtenir les horaires : elle envoie `GET /trains?from=Paris&to=Lyon&date=2026-11-03` et reçoit un JSON. Elle ne connaît ni les bases de données, ni les algorithmes de calcul d'itinéraire : elle consomme un contrat.

1.2. Rôle et importance des API

- **Interopérabilité :** faire dialoguer des technologies hétérogènes (Python, Java, JavaScript, mobile).
- **Découplage :** les équipes évoluent indépendamment tant que le contrat est respecté.

- **Réutilisation :** un service de paiement, d'authentification ou de géolocalisation est partagé par plusieurs produits.
- **Automatisation :** les API permettent l'intégration continue, les workflows, les agents et les scripts.
- **Écosystème et monétisation :** une API publique peut attirer des développeurs et générer des revenus (Stripe, Twilio, OpenWeather).
- **Scalabilité organisationnelle :** les équipes communiquent par contrats plutôt que par réunions ; c'est la base des microservices.

1.3. Domaines d'utilisation

Domaine	Usage typique de l'API
Web & mobile	Application front qui consomme une API back-end (REST/JSON).
Cloud & DevOps	API d'orchestration (Kubernetes), de provisioning (AWS), de CI/CD.
Finance / fintech	Paiement (Stripe), agrégation bancaire (DSP2), vérification KYC.
E-commerce	Catalogue, panier, stock, livraison, places de marché.
IoT	Remontée de capteurs, commande d'objets connectés (MQTT/HTTP).
Santé	Échange de dossiers patients (FHIR), prise de rendez-vous.
IA & données	Inférence de modèles, recherche, recommandation, LLM.
Interne / microservices	Communication entre services d'un même système d'information.
Géolocalisation	Cartes, itinéraires, géocodage (Google Maps, OpenStreetMap).
Communication	E-mail, SMS, notifications push, visioconférence.

1.4. Avantages et limites des API

Avantages	Limites / risques
Standardisation et documentation	Dépendance à un tiers (indisponibilité, changement de contrat)
Découplage des équipes et des technologies	Complexité de gouvernance et de versioning
Réutilisation et gain de temps	Surface d'attaque supplémentaire à sécuriser
Scalabilité horizontale	Latence réseau et surcoût de sérialisation
Écosystème, intégrations, monétisation	Quotas, coûts variables, rate limiting
Observabilité et automatisation	Nécessite logs, métriques, traces, alerting

À RETENIR

Une API n'est pas seulement technique : c'est un **produit** avec des utilisateurs (développeurs), un contrat, une documentation, un support, des versions et des engagements de service (SLA).

II Classification des API

Toutes les API ne se ressemblent pas : le style technique, la politique de publication et le contexte déterminent des choix très différents.

2.1. Les styles et technologies d'API Web

REST (Representational State Transfer)

Style architectural introduit par Roy Fielding en 2000. Il repose sur les ressources identifiées par des URIs, les méthodes HTTP et des représentations (souvent JSON). C'est le style dominant des API publiques.

Pourquoi : simplicité, compatibilité avec le Web, cache HTTP, outillage abondant.
Comment : ressources au pluriel (`/tasks`), verbes HTTP, codes de statut, HATEOAS optionnel.
Exemple : `GET /api/v1/tasks/42` renvoie `200` et `{"id":42,"title":"..."}`.

SOAP (Simple Object Access Protocol)

Protocole basé sur XML, avec une enveloppe normalisée, un contrat WSDL et de nombreuses extensions (WS-Security, WS-Addressing). Très utilisé dans les systèmes bancaires et les SI d'entreprise.

Pourquoi : contrats stricts, sécurité et transactions normalisées, interopérabilité d'entreprise.
Comment : un seul endpoint, opérations décrites dans un WSDL, messages XML.
Exemple : `POST /BankService` avec une enveloppe `<soap:Envelope>` contenant `<GetBalance>`.

GraphQL

Langage de requête et runtime créé par Facebook (2015). Le client demande exactement les champs dont il a besoin, à travers un seul endpoint.

Pourquoi : éviter le sur-fetch et le sous-fetch, agréger plusieurs sources, typage fort du schéma.
Comment : schéma (`type Query`), requêtes `query`, mutations, resolvers.
Exemple : `{ task(id: 42) { title done } }` renvoie uniquement les champs demandés.

gRPC

Framework RPC de Google basé sur HTTP/2 et Protocol Buffers, avec génération de code client/serveur et support du streaming bidirectionnel.

Pourquoi : performances élevées, contrat binaire compact, streaming, typage fort.
Comment : fichier `.proto`, génération de stubs, appels de méthodes distantes.
Exemple : `rpc GetTask(TaskRequest) returns (TaskResponse)`.

Autres styles

- **JSON-RPC / XML-RPC :** appels de procédures simples via HTTP, sans sémantique de ressources.
- **WebSocket :** canal bidirectionnel persistant, adapté au temps réel (chat, jeux, cours boursiers).
- **SSE (Server-Sent Events) :** flux unidirectionnel serveur → client, simple et compatible HTTP.
- **Webhooks :** le serveur notifie le client via un appel HTTP sortant (événements).

Style	Transport / format	Points forts	Limites	Cas d'usage
REST	HTTP / JSON	Simple, cache, outillage	Sur/sous-fetch, temps réel limité	API publiques, CRUD, mobile
SOAP	HTTP / XML	Contrats, sécurité, transactions	Verbeux, lourd, rigide	Banque, SI d'entreprise
GraphQL	HTTP / JSON	Requêtes précises, schéma typé	Cache complexe, coût des requêtes	Frontends, agrégation
gRPC	HTTP/2 / Protobuf	Rapide, compact, streaming	Moins navigateur-friendly	Microservices internes
WebSocket	TCP / binaire ou texte	Temps réel bidirectionnel	Gestion de connexions, scaling	Chat, jeux, trading
Webhooks	HTTP / JSON	Événementiel, découplé	Retries, sécurité des callbacks	Paiement, CI/CD, SaaS

2.2. Les politiques de publication des API

Politique	Public	Objectif	Exemple
Privée / interne	Équipes de l'organisation	Découpler les services, réutiliser	API de catalogue interne
Partenaire (B2B)	Partenaires identifiés	Intégrations contractuelles	API bancaire pour un assureur
Publique / ouverte	Tout développeur	Écosystème, notoriété, revenus	API météo, paiement Stripe
API produit	Clients de l'entreprise	Monétisation, self-service	API de facturation, SaaS

La politique de publication conditionne la documentation, le niveau de support, les quotas, le modèle d'authentification et les engagements de disponibilité.

2.3. Critères de choix d'une API

- **Besoins fonctionnels :** CRUD simple, requêtes complexes, temps réel, streaming, événements.
- **Contraintes techniques :** navigateur, mobile, réseau limité, latence, volume, taille des messages.
- **Sécurité :** authentification, autorisation fine, conformité (RGPD, PCI-DSS).
- **Écosystème et compétences :** langages, bibliothèques, outils de test, courbe d'apprentissage.
- **Cache et performance :** cache HTTP, CDN, compression, pagination.
- **Documentation et support :** OpenAPI, exemples, SDK, communauté.
- **Coût total :** développement, exploitation, monitoring, licences.
- **Évolutivité :** versioning, compatibilité ascendante, gouvernance.

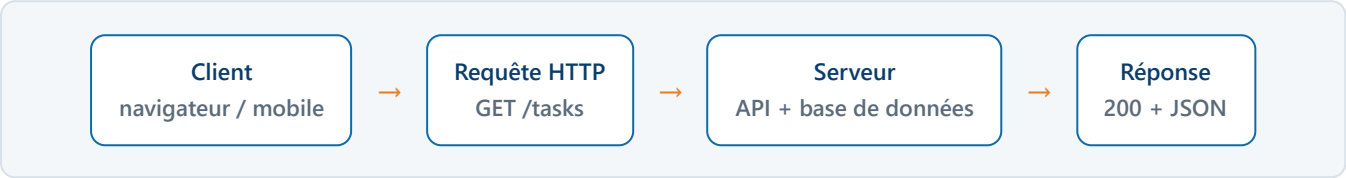
> **MÉTHODE DE CHOIX**
>
> Commencer par le besoin métier, puis éliminer les styles incompatibles avec les contraintes dures (navigateur, temps réel, sécurité), puis comparer les options restantes sur le coût et l'écosystème. Pour une API CRUD classique consommée par un front web, **REST + JSON** reste le choix par défaut le plus rationnel.

III Architecture des applications et des API

L'architecture décrit la manière dont les composants d'un système sont organisés, communiquent et évoluent. Le choix conditionne la scalabilité, la résilience et l'organisation des équipes.

3.1. Architecture client-serveur

Le **client** (navigateur, mobile, script) envoie une requête ; le **serveur** traite et renvoie une réponse. C'est le modèle de base du Web et de toute API HTTP.



Pourquoi : séparer les responsabilités, permettre des clients multiples, faire évoluer le serveur indépendamment.
Comment : définir un contrat d'API, gérer l'état côté client ou dans une base, prévoir cache et timeouts.
Exemple : une application React interroge une API FastAPI ; l'application mobile Android interroge la même API.

3.2. Architecture monolithique

Une application unique, déployée comme un seul artefact, contenant toutes les fonctionnalités (interface, logique métier, accès aux données).

- **Avantages :** simplicité de développement et de déploiement, transactions ACID locales, débogage plus simple, coût initial faible.
- **Inconvénients :** couplage fort, scaling global, déploiements risqués, difficulté à faire évoluer plusieurs équipes, dette technique.
- **Variante recommandée :** le *modular monolith* (monolithe modulaire), avec des modules bien séparés et des frontières claires, qui prépare une éventuelle extraction en services.

Exemple : une application de gestion de tâches avec interface serveur, authentification, base de données et export PDF dans un seul projet FastAPI.

3.3. Architecture orientée services (SOA)

Des services métier autonomes et réutilisables communiquent via un bus d'intégration (ESB). Les services exposent des contrats (souvent SOAP/WSDL) et sont orchestrés.

- **Avantages :** réutilisation, gouvernance centralisée, intégration de systèmes hétérogènes.
- **Inconvénients :** ESB centralisé et coûteux, complexité, couplage via le bus, latence, lourdeur de gouvernance.
- **Exemple :** un SI bancaire expose les services « Client », « Compte », « Paiement » via un ESB.

3.4. Architecture microservices

Un système composé de services petits, indépendamment déployables, organisés autour de contextes métier (*bounded contexts* du DDD), chacun avec ses données et son contrat d'API.

- **Avantages :** déploiement indépendant, scaling ciblé, autonomie des équipes, choix technologique par service, isolation des pannes.
- **Inconvénients :** complexité distribuée (réseau, latence, pannes partielles), cohérence des données (sagas), observabilité, coût opérationnel, gouvernance des contrats.
- **Patterns clés :** API Gateway, service discovery, circuit breaker, bulkhead, saga, outbox, tracing distribué.



ERREUR FRÉQUENTE

Adopter les microservices trop tôt. Une équipe de deux développeurs avec un domaine mal compris sera plus efficace avec un monolithe modulaire. Les microservices résolvent des problèmes d'*organisation* et d'*échelle*, pas des problèmes de code.

3.5. Comparaison des architectures

Critère	Monolithe	SOA	Microservices
Déploiement	Unique et simple	Services + ESB	Indépendant par service
Scaling	Global	Par service	Par service, granulaire
Couplage	Fort	Moyen (bus)	Faible (contrats)
Cohérence des données	ACID local	Transactions distribuées	Éventuelle (sagas)
Complexité opérationnelle	Faible	Élevée	Très élevée
Autonomie des équipes	Faible	Moyenne	Forte
Coût initial	Faible	Élevé	Moyen à élevé
Cas d'usage	MVP, PME, domaine simple	SI hétérogène, entreprise	Grande échelle, équipes multiples

Règle pragmatique : commencer simple (monolithe modulaire), mesurer les points de douleur, extraire des services seulement lorsqu'un besoin réel le justifie.

IV Fonctionnement d'une API Web

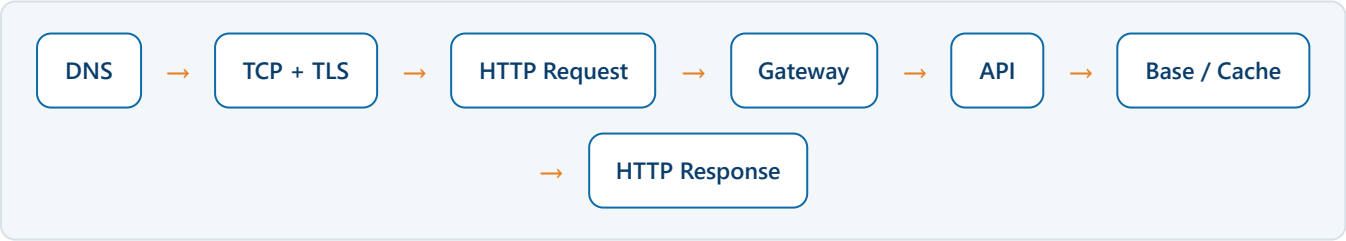
Comprendre le trajet d'une requête, du clic jusqu'à la base de données et retour, est indispensable pour diagnostiquer les erreurs, optimiser les performances et sécuriser l'ensemble.

4.1. Principes de communication sur le Web

- **URI :** identifie une ressource (`https://api.exemple.com/v1/tasks/42`).
- **DNS :** traduit le nom de domaine en adresse IP.
- **TCP :** transporte les octets de manière fiable, dans l'ordre.
- **TLS :** chiffre la connexion (HTTPS) et authentifie le serveur via un certificat.
- **HTTP :** protocole de requête/réponse, sans état entre deux requêtes.
- **Intermédiaires :** proxies, API Gateway, load balancer, CDN, reverse proxy (Nginx, Traefik).
- **Stateless :** chaque requête contient tout le contexte nécessaire (token, paramètres) ; l'état métier est stocké côté serveur (base, cache).

4.2. Processus d'appel d'une API

1. **Résolution DNS** du nom de domaine.
2. **Ouverture TCP** (handshake) puis **négociation TLS** (HTTPS).
3. **Envoi de la requête HTTP** : méthode, chemin, en-têtes, corps éventuel.
4. **Routage** par le serveur ou la passerelle vers le bon endpoint.
5. **Validation** des paramètres, de l'authentification et des autorisations.
6. **Traitement métier** et accès aux données (base, cache, services externes).
7. **Construction de la réponse** : code, en-têtes, corps sérialisé (JSON).
8. **Transport retour**, éventuel cache, puis traitement par le client.



4.3. Structure d'une requête HTTP

Une requête se compose d'une **ligne de requête**, d'**en-têtes** et d'un **corps** optionnel.

```
POST /api/v1/tasks HTTP/1.1
Host: api.exemple.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
User-Agent: MonApp/1.0
Idempotency-Key: 6f1c2a9e-...

{
  "title": "Réviser HTTP",
  "due_date": "2026-11-05",
  "priority": "high"
}
```

En-tête	Rôle	Exemple
`Host`	Nom du serveur cible (obligatoire en HTTP/1.1)	`api.exemple.com`
`Accept`	Format de réponse souhaité	`application/json`
`Content-Type`	Format du corps envoyé	`application/json`
`Authorization`	Preuve d'identité / de droits	`Bearer <token>`
`Idempotency-Key`	Éviter les doublons en cas de retry	UUID unique
`If-None-Match`	Cache conditionnel	`"etag-123"`

4.4. Structure d'une réponse HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/v1/tasks/43
Cache-Control: no-store
X-Request-Id: 9b2f7c1e

{
  "id": 43,
  "title": "Réviser HTTP",
  "done": false,
  "created_at": "2026-10-27T09:30:00Z"
}
```

En-tête	Rôle
`Content-Type`	Type MIME du corps (JSON, XML, etc.)
`Content-Length`	Taille du corps en octets
`Cache-Control`	Politique de cache (`no-store` , `max-age=60`)
`ETag`	Version de la ressource pour le cache conditionnel
`Location`	URI de la ressource créée (avec 201)
`Retry-After`	Délai conseillé après un 429 ou 503
`X-Request-Id`	Identifiant de corrélation pour le support

4.5. Formats d'échange de données

JSON

Format texte léger, lisible, natif en JavaScript. Types de base : objet, tableau, chaîne, nombre, booléen, `null`. C'est le format par défaut des API REST modernes.

```
{
  "id": 43,
  "title": "Réviser HTTP",
  "tags": ["api", "rest"],
  "done": false,
  "due_date": "2026-11-05",
  "meta": null
}
```

Bonnes pratiques : dates ISO 8601 (`2026-11-05T14:30:00Z`), identifiants explicites, pas de nombres flottants pour l'argent (utiliser des entiers en centimes ou des chaînes décimales), pas de champs `null` inutiles, versionner le schéma.

XML

Format balisé, verbeux mais très structuré, utilisé par SOAP et certains SI. Il dispose de schémas XSD et de transformations XSLT.

```
<task id="43">
  <title>Réviser HTTP</title>
  <done>false</done>
  <dueDate>2026-11-05</dueDate>
</task>
```

Autres formats

- **application/x-www-form-urlencoded :** formulaires HTML simples.
- **multipart/form-data :** upload de fichiers + champs.
- **application/octet-stream :** binaire brut (fichier, image).
- **Protocol Buffers :** binaire compact pour gRPC.
- **CSV / Parquet :** exports de données et analytics.
- **YAML :** configuration, spécifications OpenAPI.

NÉGOCIATION DE CONTENU

Le client envoie `Accept: application/json` ; le serveur répond avec `Content-Type: application/json`. S'il ne peut pas produire ce format, il renvoie `406 Not Acceptable`. FastAPI gère cela via les paramètres et les types de réponse.

V Protocole HTTP et méthodes de requête

HTTP est le langage commun du Web. Une API REST n'est rien d'autre qu'un ensemble de conventions solides posées sur HTTP.

5.1. Présentation du protocole HTTP

HTTP (*HyperText Transfer Protocol*) est un protocole de communication client-serveur, textuel dans sa version 1.1, sans état, où chaque échange est une requête suivie d'une réponse. Il est normalisé par l'IETF (RFC 9110 pour la sémantique).

Version	Transport	Apports principaux
HTTP/1.0	TCP	Une connexion par requête, en-têtes simples.
HTTP/1.1	TCP	Connexions persistantes (*keep-alive*), pipelining, cache, Host obligatoire.
HTTP/2	TCP + TLS (en pratique)	Multiplexage des flux, compression des en-têtes (HPACK), priorisation, binaire.
HTTP/3	QUIC / UDP	Réduction de la latence, reprise rapide, meilleure résistance aux pertes.

Sans état : le serveur ne mémorise pas automatiquement le contexte entre deux requêtes. L'authentification et les préférences sont donc transmises à chaque appel (token, cookies, paramètres), et l'état métier vit dans une base ou un cache.

5.2. Les méthodes HTTP

Les méthodes indiquent l'**intention** de la requête. Elles portent une sémantique normalisée : *sûre* (pas de modification) et *idempotente* (le même appel répété produit le même effet).

Méthode	Rôle	Sûre	Idempotente	Corps	Exemple
GET	Lire une ressource ou une collection	Oui	Oui	Non	`GET /tasks?done=false`
POST	Créer une ressource / déclencher un traitement	Non	Non	Oui	`POST /tasks`
PUT	Remplacer intégralement une ressource	Non	Oui	Oui	`PUT /tasks/42`
PATCH	Modifier partiellement une ressource	Non	Pas toujours	Oui	`PATCH /tasks/42`
DELETE	Supprimer une ressource	Non	Oui	Optionnel	`DELETE /tasks/42`
HEAD	Comme GET, sans le corps	Oui	Oui	Non	`HEAD /tasks/42`
OPTIONS	Découvrir les méthodes / CORS	Oui	Oui	Non	`OPTIONS /tasks`

Idempotence : pourquoi c'est essentiel

Un client peut réessayer une requête après un timeout réseau. Si l'opération est idempotente, un second appel ne crée pas de doublon et ne produit pas d'effet supplémentaire. `POST` n'étant pas idempotent, on

utilise souvent une **clé d'idempotence** : le serveur mémorise la clé et renvoie la même réponse si la requête est rejouée.

```
# Requête initiale
POST /api/v1/payments
Idempotency-Key: 8f14e45f-ea5b-4c1a-9f5e-2d7c3b6a1f00

# Retry réseau : même clé = même résultat, pas de double débit
```

Mapping CRUD ↔ HTTP ↔ SQL

Opération	HTTP	URI	SQL	Succès
Lister	GET	`/tasks`	SELECT	200
Lire	GET	`/tasks/42`	SELECT … WHERE id=42	200
Créer	POST	`/tasks`	INSERT	201
Remplacer	PUT	`/tasks/42`	UPDATE (tous champs)	200 / 204
Modifier	PATCH	`/tasks/42`	UPDATE (champs partiels)	200 / 204
Supprimer	DELETE	`/tasks/42`	DELETE	204

EXEMPLE : PUT VS PATCH

`PUT /tasks/42` avec `{"title":"A","done":true}` remplace toute la ressource : les champs omis sont réinitialisés ou rejetés. `PATCH /tasks/42` avec `{"done":true}` ne modifie que le champ `done`. Choisir PUT pour une réécriture complète, PATCH pour une mise à jour ciblée.

PIÈGES FRÉQUENTS

- Utiliser `GET` pour modifier des données (non sûr, cache et robots peuvent le déclencher).
- Mettre un corps dans `GET` : non standard, ignoré par certains proxies.
- Utiliser `POST /tasks/42/delete` au lieu de `DELETE /tasks/42`.
- Utiliser `PUT` avec un corps partiel : ce n'est pas la sémantique de PUT.

VI Codes de statut HTTP

Le code de statut est le premier contrat de lecture d'une réponse : il indique si l'appel a réussi, échoué côté client ou côté serveur, et oriente la stratégie de retry.

6.1. Présentation des codes de statut

Un code est un nombre à trois chiffres : le premier chiffre indique la **catégorie**, les deux suivants précisent le cas. Le client doit toujours vérifier le code avant de lire le corps de la réponse.

6.2. Les cinq catégories de codes HTTP

Catégorie	Signification	Codes usuels	Que doit faire le client ?
1xx	Information	100 Continue, 101 Switching Protocols	Continuer l'envoi / changer de protocole
2xx	Succès	200, 201, 202, 204, 206	Lire la réponse, mettre à jour l'interface
3xx	Redirection	301, 302, 304, 307, 308	Suivre la redirection ou utiliser le cache
4xx	Erreur du client	400, 401, 403, 404, 409, 422, 429	Corriger la requête, ne pas retenter tel quel
5xx	Erreur du serveur	500, 502, 503, 504	Retenter avec backoff, alerter, ne pas blâmer le client

Codes essentiels à connaître

Code	Nom	Quand l'utiliser
200	OK	Lecture ou mise à jour réussie avec corps.
201	Created	Création réussie ; ajouter l'en-tête `Location`.
202	Accepted	Traitement asynchrone accepté (file d'attente).
204	No Content	Succès sans corps (DELETE, PATCH vide).
206	Partial Content	Téléchargement partiel (Range).
301 / 308	Moved Permanently	Déplacement définitif ; 308 préserve la méthode.
304	Not Modified	Cache conditionnel : le client réutilise sa copie.
400	Bad Request	Requête mal formée, JSON invalide.
401	Unauthorized	Authentification absente ou invalide (en-tête `WWW-Authenticate`).
403	Forbidden	Authentifié mais non autorisé.
404	Not Found	Ressource inexistante (ou masquée volontairement).
405	Method Not Allowed	Méthode non supportée ; en-tête `Allow`.
409	Conflict	Conflit d'état (doublon, version obsolète).
415	Unsupported Media Type	Content-Type non géré.
422	Unprocessable Content	Syntaxe correcte mais validation métier échouée.
429	Too Many Requests	Quota dépassé ; en-tête `Retry-After`.
500	Internal Server Error	Bug ou exception non gérée.
502 / 504	Bad Gateway / Gateway Timeout	Passerelle en erreur ou dépendance trop lente.
503	Service Unavailable	Maintenance ou surcharge ; `Retry-After`.

6.4. Gestion des erreurs dans les API

Une API professionnelle renvoie des erreurs **cohérentes**, **documentées** et **exploitables** par le client. Le standard recommandé est **Problem Details** (RFC 9457), avec le type MIME `application/problem+json`.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://api.exemple.com/errors/validation",
  "title": "Échec de validation",
  "status": 422,
  "detail": "Le champ 'due_date' doit être une date future.",
  "instance": "/api/v1/tasks",
  "request_id": "9b2f7c1e",
  "errors": [
    {"field": "due_date", "message": "date passée", "code": "date_in_future"}
  ]
}
```

Bonnes pratiques :

- Un format d'erreur unique pour toute l'API (mêmes champs, même structure).
- Un code HTTP correct ; ne pas renvoyer 200 avec `{"error": ...}`.
- Un message utile pour le développeur, sans divulguer de secrets, de stack traces ni de détails d'infrastructure.
- Un identifiant de corrélation (`request_id`) présent dans les logs et la réponse.
- Documenter les erreurs possibles dans OpenAPI, avec des exemples.
- Retenter uniquement les erreurs transitoires (429, 502, 503, 504) avec un backoff exponentiel et un jitter.

> **COMMENT RETENTER INTELLIGEMMENT**
>
> Ne jamais retenter un `400`, `401`, `403`, `404` ou `422` sans correction. Pour `429` et `5xx`, respecter `Retry-After`, plafonner le nombre d'essais (3 à 5), et utiliser des clés d'idempotence pour les écritures.

VII Architecture REST

REST n'est pas un protocole ni une norme : c'est un style architectural défini par des contraintes. Les respecter produit des API prévisibles, cacheables et évolutives.

7.1. Définition et origine

REST (*Representational State Transfer*) a été formalisé par **Roy Fielding** dans sa thèse de doctorat en 2000. L'idée centrale : exposer des **ressources** identifiées par des URIs, que les clients manipulent à travers des **représentations** (JSON, XML...), en utilisant l'interface uniforme de HTTP.

Une API est dite *RESTful* lorsqu'elle respecte les contraintes ci-dessous. En pratique, beaucoup d'API dites « REST » sont en réalité « HTTP + JSON orientées ressources » et n'implémentent pas HATEOAS.

7.2. Les contraintes REST

Contrainte	Signification	Bénéfice
Client-serveur	Séparation de l'interface et du stockage/traitement.	Évolution indépendante des deux côtés.
Sans état	Chaque requête contient tout le contexte nécessaire.	Scalabilité, reprise après panne, simplicité.
Cacheable	Les réponses indiquent si elles peuvent être mises en cache.	Réduction de la charge et de la latence.
Interface uniforme	Identification des ressources, manipulation par représentations, messages auto-descriptifs, HATEOAS.	Prévisibilité, interopérabilité, découplage.
Système en couches	Proxies, gateways, CDN, load balancers intercalés.	Sécurité, scalabilité, résilience.
Code à la demande (optionnel)	Le serveur peut envoyer du code exécutable (JS).	Extensibilité du client.

7.3. Ressources, URIs et représentations

- **Ressource :** un objet ou une collection métier (`task`, `tasks`, `users`).
- **URI :** un nom de ressource, pas une action. On écrit `/tasks`, pas `/getTasks`.
- **Représentation :** la forme transmise à un instant donné (JSON), distincte de la ressource elle-même.
- **Nommage :** noms au pluriel, en minuscules, séparés par des tirets (`/task-lists`), pas de verbes.
- **Hiérarchie :** `/users/42/tasks` pour les tâches d'un utilisateur ; éviter les imbrications profondes (2 niveaux maximum).

Exemple de conception pour l'API Tâches

Action	Méthode + URI	Réponse
Lister les tâches	`GET /api/v1/tasks?done=false&limit=20&offset=0`	200 + tableau + métadonnées de pagination
Lire une tâche	`GET /api/v1/tasks/42`	200 + objet / 404
Créer une tâche	`POST /api/v1/tasks`	201 + objet + `Location`
Remplacer une tâche	`PUT /api/v1/tasks/42`	200 + objet / 404
Modifier partiellement	`PATCH /api/v1/tasks/42`	200 + objet / 404
Supprimer	`DELETE /api/v1/tasks/42`	204 / 404
Sous-ressource	`GET /api/v1/tasks/42/comments`	200 + commentaires

7.4. HATEOAS et maturité de Richardson

HATEOAS (*Hypermedia As The Engine Of Application State*) consiste à inclure dans la réponse les liens vers les actions possibles. Le client découvre les transitions au lieu de coder des URLs en dur.

```
{
  "id": 42,
  "title": "Réviser REST",
  "done": false,
  "_links": {
    "self":   {"href": "/api/v1/tasks/42"},
    "update": {"href": "/api/v1/tasks/42", "method": "PATCH"},
    "delete": {"href": "/api/v1/tasks/42", "method": "DELETE"},
    "owner":  {"href": "/api/v1/users/7"}
  }
}
```

Le **modèle de maturité de Richardson** distingue quatre niveaux :

Niveau	Caractéristique	Exemple
0	Un seul endpoint, style RPC	`POST /api` avec une action dans le corps
1	Ressources identifiées par des URIs	`/tasks/42`
2	Verbes HTTP + codes de statut	`GET`, `POST`, `404`
3	HATEOAS	Liens d'actions dans la réponse

La majorité des API publiques se situent au niveau 2. HATEOAS apporte de la découvrabilité mais complexifie le client et reste peu utilisé en pratique.

7.5. Bonnes pratiques de conception REST

- **Versionner** l'API : `/api/v1/...` ou en-tête `Accept: application/vnd.api.v1+json`.
- **Pagination** systématique des collections : `limit` / `offset` ou curseur (`cursor`), avec métadonnées.
- **Filtrage, tri, recherche** via query params : `?status=open&sort=-created_at&q=urgent`.

- **Sparse fieldsets** si nécessaire : `?fields=id,title,done`.
- **Codes de statut corrects** et erreurs normalisées.
- **Idempotence** des écritures sensibles via `Idempotency-Key`.
- **Cache** : `ETag`, `Cache-Control`, `Last-Modified`.
- **Sécurité** : HTTPS, authn/authz, validation, rate limiting.
- **Documentation** OpenAPI à jour et exemples réalistes.
- **Compatibilité** : ajouter des champs est non cassant ; en supprimer ou renommer est cassant.

EXEMPLE DE PAGINATION

```
GET /api/v1/tasks?limit=20&offset=40

{
  "items": [ ... ],
  "total": 137,
  "limit": 20,
  "offset": 40,
  "next": "/api/v1/tasks?limit=20&offset=60",
  "prev": "/api/v1/tasks?limit=20&offset=20"
}
```

VIII Documentation des API

Une API sans documentation est une API que personne n'utilise. La documentation est le contrat visible, la porte d'entrée de l'écosystème et le support de première ligne.

8.1. Importance de la documentation

- **Contrat :** elle fige les formats, les erreurs et les règles attendues.
- **Onboarding :** un développeur peut intégrer l'API sans appeler l'équipe.
- **Réduction du support :** les questions fréquentes ont une réponse écrite.
- **Tests et génération :** elle permet de générer des clients, des mocks et des tests.
- **Gouvernance :** les évolutions cassantes sont identifiées et versionnées.
- **Découvrabilité :** une API documentée est adoptée plus vite.

8.2. OpenAPI

OpenAPI (anciennement Swagger Specification) est un standard ouvert pour décrire une API HTTP de façon lisible par un humain et une machine. La version actuelle est **OpenAPI 3.1**, alignée sur JSON Schema 2020-12.

Une spécification décrit : les métadonnées (`info`), les serveurs, les chemins et opérations, les paramètres, les corps de requête, les réponses, les schémas réutilisables (`components`) et les schémas de sécurité.

```yaml
openapi: 3.1.0
info:
  title: API Tâches
  version: 1.0.0
  description: Gestion de tâches - exemple de veille
servers:
  - url: https://api.exemple.com/api/v1
paths:
  /tasks:
    get:
      summary: Lister les tâches
      parameters:
        - name: done
          in: query
          schema: {type: boolean}
        - name: limit
          in: query
          schema: {type: integer, minimum: 1, maximum: 100, default: 20}
      responses:
        "200":
          description: Liste paginée
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/TaskList"
    post:
      summary: Créer une tâche
      requestBody:
        required: true
        content:
          application/json:
            schema: {$ref: "#/components/schemas/TaskCreate"}
      responses:
        "201":
          description: Tâche créée
          content:
            application/json:
              schema: {$ref: "#/components/schemas/Task"}
components:
  schemas:
    Task:
      type: object
      required: [id, title, done]
      properties:
        id: {type: integer, format: int64}
        title: {type: string, minLength: 1, maxLength: 200}
        done: {type: boolean, default: false}
        due_date: {type: string, format: date}
    TaskCreate:
      type: object
      required: [title]
      properties:
        title: {type: string, minLength: 1, maxLength: 200}
        due_date: {type: string, format: date}
```

8.3. Swagger UI et ReDoc

Swagger UI transforme une spécification OpenAPI en documentation interactive : on peut parcourir les endpoints, voir les schémas et exécuter des requêtes (« Try it out »). **ReDoc** propose une lecture plus éditoriale, adaptée à la consultation.

Avec FastAPI, ces interfaces sont générées automatiquement à partir du code :

URL	Contenu
`/docs`	Swagger UI interactif
`/redoc`	ReDoc (documentation lisible)
`/openapi.json`	Schéma OpenAPI brut (JSON)

8.5. Bonnes pratiques de documentation

- **Une source de vérité :** la spécification OpenAPI générée depuis le code, versionnée avec lui.
- **Des exemples réalistes** pour chaque endpoint, chaque erreur et chaque schéma.
- **Documenter l'authentification** (comment obtenir un token, quels scopes).
- **Documenter les erreurs** : codes, format, causes, remèdes.
- **Un guide de démarrage** (quickstart) et un guide des concepts.
- **Un changelog** et une politique de versioning claire.
- **Documentation-as-code** : relue en pull request, testée en CI (linting Spectral, validation du schéma).
- **SDK et collections Postman** générés depuis la spécification.
- **Accessibilité** : français/anglais selon le public, recherche, navigation claire.

> **CHECKLIST DOCUMENTATION**
>
> Quickstart · authentification · endpoints · schémas · erreurs · pagination · rate limits · versioning · exemples cURL · statut de service · contact support · conditions d'utilisation.

IX Introduction au framework FastAPI

FastAPI est un framework Python moderne, rapide et typé, qui génère automatiquement la validation et la documentation. C'est un excellent choix pour apprendre et pour produire.

9.1. Pourquoi FastAPI ?

- **Performance :** bâti sur Starlette (ASGI) et Pydantic v2 (cœur Rust), proche de Node.js et Go sur les cas simples.
- **Productivité :** les types Python servent à la fois de validation, de sérialisation et de documentation.
- **Moins de bugs :** validation automatique des entrées, erreurs claires.
- **Documentation automatique :** Swagger UI et ReDoc générés depuis le code.
- **Asynchrone natif :** `async def` et `await` pour les I/O.
- **Standards :** OpenAPI 3.1 et JSON Schema.
- **Écosystème :** dépendances, sécurité OAuth2/JWT, tests, SQLModel/SQLAlchemy.

9.2. Installation et premier lancement

```
# Créer un environnement virtuel
python -m venv .venv
.venv\Scripts\activate        # Windows
# source .venv/bin/activate   # macOS / Linux

# Installer FastAPI + serveur + CLI
pip install "fastapi[standard]"

# Lancer en développement (rechargement automatique)
fastapi dev main.py
# ou
uvicorn main:app --reload --port 8000
```

La documentation interactive est alors disponible sur `http://127.0.0.1:8000/docs`.

9.3. Structure d'un projet

```
app/
  __init__.py
  main.py            # création de l'application
  config.py          # settings (pydantic-settings)
  models/            # modèles Pydantic / ORM
    task.py
  routers/           # routes par domaine
    tasks.py
    auth.py
  dependencies/      # dépendances partagées
    db.py
    security.py
  services/          # logique métier
  tests/
    test_tasks.py
requirements.txt
Dockerfile
```

9.4. Une API CRUD complète avec FastAPI

L'exemple ci-dessous implémente l'API Tâches en mémoire (remplaçable par une base de données). Il illustre les routes, les paramètres, Pydantic, les codes de statut, les erreurs et la documentation automatique.

```python
from datetime import date
from typing import Annotated

from fastapi import FastAPI, HTTPException, Path, Query, status
from pydantic import BaseModel, Field, ConfigDict

app = FastAPI(
    title="API Tâches",
    version="1.0.0",
    description="API de démonstration pour la veille API/REST",
)

# ---------- Schémas Pydantic ----------
class TaskBase(BaseModel):
    title: str = Field(min_length=1, max_length=200, examples=["Réviser HTTP"])
    due_date: date | None = None

class TaskCreate(TaskBase):
    pass

class TaskUpdate(BaseModel):
    title: str | None = Field(default=None, min_length=1, max_length=200)
    done: bool | None = None
    due_date: date | None = None

class Task(TaskBase):
    model_config = ConfigDict(from_attributes=True)
    id: int
    done: bool = False
    created_at: date

# ---------- "Base de données" en mémoire ----------
tasks: dict[int, Task] = {}
next_id = 1

# ---------- Routes ----------
@app.get("/api/v1/tasks", response_model=list[Task], tags=["tasks"])
def list_tasks(
    done: Annotated[bool | None, Query(description="Filtrer par état")] = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    items = list(tasks.values())
    if done is not None:
        items = [t for t in items if t.done == done]
    return items[offset:offset + limit]

@app.get("/api/v1/tasks/{task_id}", response_model=Task, tags=["tasks"])
def get_task(task_id: Annotated[int, Path(ge=1)]):
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail="Tâche introuvable")
    return tasks[task_id]

@app.post("/api/v1/tasks", response_model=Task,
          status_code=status.HTTP_201_CREATED, tags=["tasks"])
def create_task(payload: TaskCreate):
    global next_id
    task = Task(id=next_id, created_at=date.today(), **payload.model_dump())
    tasks[next_id] = task
    next_id += 1
```

```
    return task

@app.patch("/api/v1/tasks/{task_id}", response_model=Task, tags=["tasks"])
def update_task(task_id: int, payload: TaskUpdate):
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail="Tâche introuvable")
    current = tasks[task_id]
    updated = current.model_copy(update=payload.model_dump(exclude_unset=True))
    tasks[task_id] = updated
    return updated

@app.delete("/api/v1/tasks/{task_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["tasks"])
def delete_task(task_id: int):
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail="Tâche introuvable")
    del tasks[task_id]
```

CE QUE FASTAPI FAIT GRATUITEMENT

- Validation des types et des contraintes (`min_length`, `ge`, `le`).
- Réponses `422` détaillées en cas de données invalides.
- Documentation Swagger UI et ReDoc, avec schémas et exemples.
- Sérialisation JSON automatique (dates, modèles Pydantic).
- Injection de dépendances et gestion des erreurs.

9.5. Validation, réponses et erreurs

```
# Réponse typée : FastAPI filtre les champs non déclarés
@app.get("/me", response_model=UserPublic)
def me(user: User = Depends(get_current_user)):
    return user

# Erreur métier explicite
raise HTTPException(
    status_code=status.HTTP_409_CONFLICT,
    detail="Une tâche avec ce titre existe déjà",
    headers={"X-Error-Code": "task_duplicate"},
)

# Gestion globale d'exceptions
@app.exception_handler(ValueError)
async def value_error_handler(request, exc):
    return JSONResponse(status_code=400, content={"detail": str(exc)})
```

9.6. Dépendances (injection)

Une dépendance est une fonction exécutée avant l'endpoint, dont le résultat est injecté. Elle sert à factoriser l'authentification, l'accès à la base, la pagination ou les permissions.

```python
from fastapi import Depends, Header

def get_db():
    db = SessionLocal()
    try:
        yield db          # code exécuté après la réponse
    finally:
        db.close()

def get_current_user(
    authorization: Annotated[str, Header()],
    db: Session = Depends(get_db),
) -> User:
    token = authorization.removeprefix("Bearer ").strip()
    user = decode_and_verify(token, db)
    if not user:
        raise HTTPException(401, "Token invalide")
    return user

@app.get("/api/v1/me")
def read_me(user: User = Depends(get_current_user)):
    return user
```

Les dépendances se composent, se surchargent dans les tests (`app.dependency_overrides`) et peuvent être asynchrones.

9.7. Organisation avec APIRouter

```python
# routers/tasks.py
from fastapi import APIRouter
router = APIRouter(prefix="/api/v1/tasks", tags=["tasks"])

@router.get("")
def list_tasks(): ...

# main.py
from app.routers import tasks
app.include_router(tasks.router)
```

9.8. Tests avec TestClient et pytest

```python
# tests/test_tasks.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_and_get_task():
    r = client.post("/api/v1/tasks", json={"title": "Test"})
    assert r.status_code == 201
    task_id = r.json()["id"]

    r = client.get(f"/api/v1/tasks/{task_id}")
    assert r.status_code == 200
    assert r.json()["title"] == "Test"

def test_validation_error():
    r = client.post("/api/v1/tasks", json={"title": ""})
    assert r.status_code == 422
```

BONNES PRATIQUES FASTAPI

- Découper en routers par domaine, garder `main.py` mince.
- Utiliser des modèles Pydantic distincts pour l'entrée, la mise à jour et la sortie.
- Ne jamais exposer directement les modèles ORM (fuite de champs sensibles).
- Configurer CORS explicitement, pas avec `*` en production.
- Centraliser la configuration avec `pydantic-settings` et des variables d'environnement.
- Tester les cas nominaux, les erreurs et les autorisations.

X Programmation synchrone et asynchrone

Une API passe l'essentiel de son temps à attendre : réseau, base de données, fichiers, services externes. Le mode d'exécution détermine sa capacité à servir plusieurs requêtes sans bloquer.

10.1. Programmation synchrone

Le code s'exécute **ligne après ligne**. Un appel bloquant (lecture disque, requête SQL, appel HTTP) immobilise le thread jusqu'à la fin de l'opération. Pour traiter plusieurs requêtes, le serveur utilise plusieurs threads ou processus.

```
import requests

def get_weather(city: str) -> dict:
    # Bloque le thread jusqu'à la réponse (plusieurs centaines de ms)
    response = requests.get(f"https://api.meteo.example/{city}", timeout=5)
    response.raise_for_status()
    return response.json()

def handler():
    data = get_weather("Lyon")   # rien d'autre ne s'exécute ici
    return data
```

Avantages : simplicité, compatibilité avec la plupart des bibliothèques, débogage facile.
Inconvénients : consommation de threads, mauvaise utilisation des I/O, saturation sous forte concurrence.

10.2. Programmation asynchrone

Le code utilise une **boucle d'événements** (*event loop*). Quand une opération d'I/O est lancée avec `await`, le contrôle est rendu à la boucle qui peut exécuter d'autres tâches ; le résultat est repris lorsque l'opération est terminée. Un seul thread peut ainsi gérer des milliers de connexions en attente.

```
import asyncio
import httpx

async def get_weather(city: str) -> dict:
    # Non bloquant : la boucle peut servir d'autres requêtes pendant l'attente
    async with httpx.AsyncClient(timeout=5) as client:
        response = await client.get(f"https://api.meteo.example/{city}")
        response.raise_for_status()
        return response.json()

async def handler():
    # Trois appels exécutés en parallèle
    lyon, paris, nice = await asyncio.gather(
        get_weather("Lyon"),
        get_weather("Paris"),
        get_weather("Nice"),
    )
    return {"lyon": lyon, "paris": paris, "nice": nice}
```

Avantages : excellent pour les I/O et les connexions nombreuses, faible empreinte mémoire, latence réduite.
Inconvénients : complexité, écosystème de bibliothèques inégal, risque de bloquer la boucle avec du code synchrone ou du CPU intensif.

10.3. Comparaison entre synchrone et asynchrone

Critère	Synchrone	Asynchrone
Modèle	Un thread par tâche (ou pool)	Une boucle d'événements, tâches coopératives
I/O	Bloquant	Non bloquant (`await`)
Concurrence	Limitée par les threads	Très élevée
Complexité	Faible	Moyenne à élevée
CPU intensif	Adapté (processus/threads)	À déporter (process pool)
Bibliothèques	Très large	Nécessite des versions async
Cas idéal	Scripts, CPU, faible concurrence	API, web, appels réseau, streaming

10.4. Async dans FastAPI

FastAPI est construit sur ASGI et supporte les deux styles :

- `async def` : exécuté dans la boucle d'événements. Utiliser des bibliothèques async (`httpx` , `asyncpg` , `motor`).
- `def` : exécuté dans un **threadpool** géré par AnyIO. Adapté aux bibliothèques synchrones (`requests` , `psycopg2`).

```
@app.get("/sync")
def sync_endpoint():
    # OK : FastAPI l'exécute dans un threadpool
    return requests.get("https://api.exemple.com/data").json()

@app.get("/async")
async def async_endpoint():
    # OK : bibliothèque async
    async with httpx.AsyncClient() as client:
        r = await client.get("https://api.exemple.com/data")
    return r.json()

@app.get("/bad")
async def bad_endpoint():
    # DANGER : requests est bloquant et gèle la boucle d'événements
    return requests.get("https://api.exemple.com/data").json()
```

LES PIÈGES DE L'ASYNC

- Appeler une bibliothèque bloquante dans `async def` : toute l'API ralentit.
- Faire du calcul lourd (image, cryptographie, ML) dans la boucle : déporter dans un processus.
- Oublier `await` : la coroutine n'est jamais exécutée.
- Créer une session/base par requête sans la fermer : fuite de ressources.
- Mélanger sync et async sans comprendre le threadpool : comportements imprévisibles.

RÈGLE DE DÉCISION SIMPLE

I/O et forte concurrence → `async def` + bibliothèques async. Bibliothèque uniquement synchrone ou code CPU → `def` (threadpool) ou processus séparé. En cas de doute, mesurer avec un test de charge (Locust, k6).

XI Sécurité des API : authentification et autorisation

Sécuriser une API consiste d'abord à répondre à deux questions : qui appelle (authentification) et a-t-il le droit de le faire (autorisation).

11.1. Principes fondamentaux

- **Confidentialité, intégrité, disponibilité (CIA) :** protéger les données, garantir qu'elles ne sont pas altérées, maintenir le service.
- **Moindre privilège :** chaque utilisateur, service ou token ne reçoit que les droits nécessaires.
- **Défense en profondeur :** plusieurs couches (TLS, auth, validation, rate limiting, logs, WAF).
- **Zéro confiance :** ne jamais faire confiance à l'entrée client, même interne.
- **Échec sécurisé :** en cas d'erreur, refuser l'accès plutôt que l'accorder.
- **Secrets hors du code :** variables d'environnement, coffre-fort (Vault), rotation.
- **Journalisation et audit :** tracer les accès sensibles sans stocker de secrets.
- **Validation systématique :** types, tailles, formats, listes blanches.

OWASP API Security Top 10 (2023)

#	Risque	Exemple / parade
API1	Broken Object Level Authorization (BOLA)	`/tasks/42` accessible à un autre utilisateur → vérifier la propriété.
API2	Broken Authentication	Tokens faibles, pas d'expiration → OAuth2/JWT, MFA, rotation.
API3	Broken Object Property Level Authorization	Champ `role` modifiable → listes blanches de champs.
API4	Unrestricted Resource Consumption	Pagination absente → quotas, limites de taille, timeouts.
API5	Broken Function Level Authorization	`/admin` accessible → contrôle de rôle par endpoint.
API6	Unrestricted Access to Sensitive Business Flows	Création massive de comptes → anti-bot, rate limiting.
API7	Server Side Request Forgery (SSRF)	URL fournie par le client → liste blanche de domaines.
API8	Security Misconfiguration	CORS `*`, debug activé → durcissement, en-têtes de sécurité.
API9	Improper Inventory Management	Versions oubliées → inventaire, dépréciation, monitoring.
API10	Unsafe Consumption of APIs	Faire confiance à une API tierce → valider, timeout, isoler.

11.2. Mécanismes d'authentification

Mécanisme	Principe	Usage	Limites
Basic Auth	Login:mot de passe encodé en Base64	Outils internes, tests	Uniquement avec TLS ; pas de logout ; peu granulaire
Clé API	Secret dans un en-tête (`X-API-Key`)	APIs serveur à serveur	Rotation, fuite, pas d'identité utilisateur
Session / cookie	Cookie signé + session serveur	Applications web	CSRF, gestion de session, scaling
Bearer token	Token opaque ou JWT dans `Authorization`	APIs, mobile, SPA	Vol de token, expiration, révocation
OAuth2 / OIDC	Délégation d'autorisation, identité fédérée	Google, GitHub, SSO	Complexité des flows, configuration
mTLS	Certificat client + serveur	Service mesh, B2B	Gestion des certificats

11.3. OAuth2

OAuth2 (RFC 6749) est un cadre d'**autorisation déléguée** : une application (client) obtient un accès limité à une ressource au nom d'un utilisateur, sans connaître son mot de passe. **OpenID Connect (OIDC)** ajoute l'authentification et le profil via un jeton d'identité (ID token).

Les rôles

- **Resource Owner :** l'utilisateur qui possède les données.
- **Client :** l'application qui demande l'accès.
- **Authorization Server :** délivre les tokens après authentification et consentement.
- **Resource Server :** l'API qui protège les ressources et valide les tokens.

Les flows principaux

Flow	Contexte	Principe
Authorization Code + PKCE	SPA, mobile, web	Redirection, code échangé contre token ; PKCE protège le code.
Client Credentials	Serveur à serveur	Le client s'authentifie directement, pas d'utilisateur.
Device Code	TV, CLI, objets connectés	L'utilisateur valide sur un autre appareil.
Refresh Token	Tous les flows avec utilisateur	Renouveler l'access token sans redemander le mot de passe.
Implicit / Password	Dépréciés	À éviter (fuite de token, stockage du mot de passe).

Exemple de flow Authorization Code + PKCE

1. Le client génère un `code_verifier` aléatoire et son empreinte `code_challenge`.
2. Redirection vers l'Authorization Server avec `client_id`, `redirect_uri`, `scope`, `state`, `code_challenge`.
3. L'utilisateur s'authentifie et consent ; le serveur redirige avec un `code`.
4. Le client échange le code + `code_verifier` contre un `access_token` (et souvent un `refresh_token`).

5. Le client appelle l'API avec `Authorization: Bearer <access_token>`.
6. L'API valide la signature, l'expiration, l'audience, l'émetteur et les scopes.

```
POST /oauth/token HTTP/1.1
Host: auth.exemple.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https://app.exemple.com/callback
&client_id=mon-app
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

11.4. JSON Web Token (JWT)

Un **JWT** (RFC 7519) est un jeton compact et signé, composé de trois parties encodées en Base64URL et séparées par des points :

```
header.payload.signature

# Header
{"alg": "RS256", "typ": "JWT", "kid": "2026-10"}

# Payload (claims)
{
  "iss": "https://auth.exemple.com",
  "sub": "user-42",
  "aud": "api-taches",
  "exp": 1793520000,
  "iat": 1793516400,
  "scope": "tasks:read tasks:write",
  "role": "user",
  "jti": "8f14e45f-..."
}
```

Signature : garantit l'intégrité et l'authenticité, pas la confidentialité. Le payload est lisible par tous : ne jamais y mettre de secret, de mot de passe ou de donnée personnelle sensible.

Algorithme	Type	Usage
HS256	Symétrique (secret partagé)	Un seul émetteur et un seul vérificateur ; simple mais secret partagé.
RS256 / RS512	Asymétrique (RSA)	Émetteur signe avec la clé privée, API vérifie avec la clé publique (JWKS).
ES256	Asymétrique (ECDSA)	Signatures plus courtes, performances intéressantes.

Validation d'un JWT côté API

1. Récupérer le token dans l'en-tête `Authorization: Bearer ...`.
2. Vérifier la signature avec la clé appropriée (JWKS pour les clés asymétriques).
3. Vérifier `exp` (expiration), `nbf` (pas avant), `iss` (émetteur) et `aud` (audience).
4. Vérifier les scopes/rôles requis pour l'opération.
5. Refuser les algorithmes non attendus (jamais `none`), limiter le *clock skew*.

```python
# Exemple avec PyJWT
import jwt
from jwt import InvalidTokenError

PUBLIC_KEY = load_public_key()

def decode_token(token: str) -> dict:
    try:
        return jwt.decode(
            token,
            PUBLIC_KEY,
            algorithms=["RS256"],
            audience="api-taches",
            issuer="https://auth.exemple.com",
            options={"require": ["exp", "iat", "iss", "aud"]},
        )
    except InvalidTokenError as exc:
        raise HTTPException(status_code=401, detail="Token invalide") from exc
```

Bonnes pratiques JWT

- **Access token court** (5 à 15 min) et **refresh token** plus long, avec rotation et révocation.
- Stocker les tokens de manière sûre : cookies `HttpOnly` + `Secure` + `SameSite` pour le web, stockage sécurisé natif pour mobile.
- Ne pas stocker de données sensibles dans le payload.
- Valider strictement `alg`, `iss`, `aud`, `exp` ; utiliser les JWKS et le `kid`.
- Prévoir une liste de révocation ou un `jti` pour les tokens critiques.
- Utiliser HTTPS partout ; jamais de token dans l'URL (fuite par logs et referer).

> **VULNÉRABILITÉS JWT CLASSIQUES**
>
> - **Algorithme `none` accepté** : toujours imposer une liste d'algorithmes.
> - **Confusion de clé** (HS256 vérifié avec une clé publique) : séparer les types de clés.
> - **Absence de vérification de l'audience** : un token destiné à un autre service est accepté.
> - **Expiration trop longue** : un token volé reste valide des jours.
> - **Stockage en `localStorage`** : exposé au XSS ; préférer des cookies sécurisés.

11.5. Autorisation et bonnes pratiques

- **RBAC** (rôles) ou **ABAC** (attributs) selon la granularité nécessaire.
- Vérifier la propriété de chaque objet (BOLA) : un utilisateur ne lit que ses tâches.
- Contrôler les fonctions sensibles (admin) séparément des objets.
- Limiter le débit (rate limiting) et la taille des requêtes.
- Configurer CORS avec une liste blanche d'origines.
- Ajouter les en-têtes de sécurité : `Strict-Transport-Security`, `X-Content-Type-Options`, `Content-Security-Policy` (pour les interfaces).
- Journaliser les événements d'authentification et les accès sensibles.
- Faire des revues de sécurité et des tests d'intrusion ; scanner les dépendances (`pip-audit`).

XII Plateformes de déploiement open source

Déployer une API, c'est la rendre accessible, supervisée et reproductible. Les PaaS auto-hébergés open source offrent une expérience proche de Heroku, sur ses propres serveurs.

12.1. Trois plateformes open source

Plateforme	Modèle	Licence	Points forts
Coolify	PaaS auto-hébergé, Docker	Apache-2.0	Interface web complète, Git deploy, Docker Compose, bases de données, SSL automatique, previews.
Dokku	Mini-Heroku, Docker	MIT	Très léger, `git push` deploy, buildpacks, plugins, idéal pour un serveur unique.
CapRover	PaaS sur Docker Swarm	Apache-2.0	Interface web, catalogue d'applications one-click, clustering Swarm, Nginx et SSL.

Coolify

Pourquoi : l'alternative open source la plus complète à Heroku/Vercel, avec une interface moderne et une communauté active.
Comment : installer sur un VPS via le script officiel, connecter un dépôt Git, définir les variables d'environnement, activer le domaine et le SSL.
Exemple : déployer le dépôt `api-taches` ; Coolify construit l'image Docker, lance le conteneur et configure Traefik avec un certificat Let's Encrypt.

Dokku

Pourquoi : minimaliste, rapide à installer, parfait pour un projet pédagogique ou une petite production sur un seul serveur.
Comment : installer sur Ubuntu, ajouter une clé SSH, créer l'application, ajouter un remote Git et pousser.
Exemple :

```
dokku apps:create api-taches
git remote add dokku dokku@mon-serveur:api-taches
git push dokku main
dokku config:set api-taches DATABASE_URL=postgres://...
```

CapRover

Pourquoi : interface graphique simple, catalogue d'applications (PostgreSQL, Redis, etc.), clustering via Docker Swarm.
Comment : installer sur un serveur, se connecter à l'interface, créer une application, configurer le déploiement Git ou l'image Docker, activer HTTPS.
Exemple : créer l'application `api-taches`, ajouter un domaine, activer « Force HTTPS », déployer depuis GitHub.

Critère	Coolify	Dokku	CapRover
Interface	Web complète	CLI	Web
Courbe d'apprentissage	Moyenne	Faible	Faible
Docker Compose	Oui	Non (plugins)	Partiel
Multi-serveurs	Oui (limité)	Via k3s	Oui (Swarm)
SSL automatique	Oui	Oui (plugin)	Oui
Idéal pour	Projets complets	Petit serveur, Git push	Débutants, catalogue

12.2. Déployer FastAPI avec Docker

Pourquoi Docker : garantir le même environnement du poste de développement à la production, isoler les dépendances et faciliter le déploiement sur n'importe quelle plateforme.

```
# Dockerfile
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["fastapi", "run", "app/main.py", "--host", "0.0.0.0", "--port", "8000"]
```

```yaml
# docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql+psycopg://app:secret@db:5432/app
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

Checklist de mise en production

- ☐ Variables d'environnement et secrets hors du dépôt Git.
- ☐ HTTPS obligatoire, redirection HTTP → HTTPS.
- ☐ Reverse proxy (Traefik, Nginx, Caddy) et en-têtes de sécurité.
- ☐ Health check `/health` et redémarrage automatique.
- ☐ Logs structurés, métriques, traces et alerting.
- ☐ Limites de ressources (CPU, mémoire) et rate limiting.
- ☐ Sauvegardes de la base de données et test de restauration.
- ☐ CI/CD : tests, lint, build, déploiement reproductible.
- ☐ Documentation de l'API à jour et versionnée.
- ☐ Plan de rollback et gestion des migrations de schéma.

EXEMPLE DE HEALTH CHECK FASTAPI

```python
@app.get("/health", tags=["ops"])
def health():
    return {"status": "ok", "version": app.version}
```

XIII Références

Sources officielles, normes et ressources utilisées pour ce travail de veille. Les cheatSheets du dossier s'appuient sur ces mêmes sources.

Normes et protocoles

- RFC 9110 — HTTP Semantics : https://www.rfc-editor.org/rfc/rfc9110
- RFC 9111 — HTTP Caching : https://www.rfc-editor.org/rfc/rfc9111
- RFC 9112 / 9113 / 9114 — HTTP/1.1, HTTP/2, HTTP/3
- RFC 9457 — Problem Details for HTTP APIs : https://www.rfc-editor.org/rfc/rfc9457
- RFC 6749 — OAuth 2.0 : https://www.rfc-editor.org/rfc/rfc6749
- RFC 7636 — PKCE : https://www.rfc-editor.org/rfc/rfc7636
- RFC 7519 — JSON Web Token : https://www.rfc-editor.org/rfc/rfc7519
- OpenAPI Specification 3.1 : https://spec.openapis.org/oas/v3.1.0.html
- MDN Web Docs — HTTP : https://developer.mozilla.org/fr/docs/Web/HTTP

FastAPI et écosystème Python

- Documentation FastAPI : https://fastapi.tiangolo.com/
- Tutoriel FastAPI : https://fastapi.tiangolo.com/tutorial/
- Pydantic : https://docs.pydantic.dev/
- Starlette : https://www.starlette.io/
- Uvicorn : https://www.uvicorn.org/
- FastAPI Cheat Sheet (Real Python) : https://realpython.com/cheatsheets/fastapi/
- SQLModel : https://sqlmodel.tiangolo.com/
- pytest : https://docs.pytest.org/

Sécurité

- OWASP API Security Top 10 (2023) : https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- OWASP Cheat Sheet Series : https://cheatsheetseries.owasp.org/
- OAuth 2.0 Security Best Current Practice : draft-ietf-oauth-security-topics
- JWT.io : https://jwt.io/
- PyJWT : https://pyjwt.readthedocs.io/

Déploiement et DevOps

- Coolify : https://coolify.io/ — github.com/coollabsio/coolify
- Dokku : https://dokku.com/ — github.com/dokku/dokku
- CapRover : https://caprover.com/ — github.com/caprover/caprover
- Docker : https://docs.docker.com/
- Kubernetes : https://kubernetes.io/fr/docs/
- Postman Learning Center : https://learning.postman.com/
- VSCode : https://code.visualstudio.com/docs

CheatSheets utilisés

- Awesome Cheatsheets : https://github.com/LeCoupa/awesome-cheatsheets
- HTTP Status Codes : https://httpstatuses.io/
- MDN HTTP Status : https://developer.mozilla.org/fr/docs/Web/HTTP/Status
- OWASP Cheat Sheets : https://cheatsheetseries.owasp.org/
- Real Python FastAPI Cheat Sheet : https://realpython.com/cheatsheets/fastapi/

Les cheatSheets fournis dans le dossier `cheatsheets/` sont des synthèses originales réalisées à partir de ces sources ; ils ne copient pas les documents tiers.

✓ Conclusion et aide-mémoire

Une API réussie est un produit : un contrat clair, un protocole bien utilisé, une architecture adaptée, une sécurité sérieuse et un déploiement reproductible.

Synthèse en dix points

1. Une **API** est un contrat d'interopérabilité, pas seulement du code.
2. **REST** = ressources + méthodes HTTP + représentations + sans état + cache.
3. Choisir le **style** (REST, GraphQL, gRPC, SOAP) selon le besoin, pas par mode.
4. L'**architecture** doit suivre l'organisation et l'échelle, pas l'inverse.
5. **HTTP** fournit la sémantique : méthodes, codes, en-têtes, cache.
6. Des **erreurs normalisées** (RFC 9457) réduisent le coût d'intégration.
7. **OpenAPI + Swagger UI** rendent l'API découvrable et testable.
8. **FastAPI** combine types, validation, documentation et performance.
9. **OAuth2/JWT** : authentifier, autoriser, valider, expirer, révoquer.
10. **Docker + Coolify/Dokku/CapRover** rendent le déploiement reproductible.

Checklist finale du projet

- ☐ Contrat OpenAPI versionné et documentation à jour.
- ☐ Validation des entrées et erreurs normalisées.
- ☐ Authentification et autorisation testées (BOLA, admin, scopes).
- ☐ Rate limiting, quotas et limites de taille.
- ☐ Logs corrélés, métriques et health check.
- ☐ Tests unitaires et d'intégration, CI verte.
- ☐ Dockerfile reproductible et variables d'environnement documentées.
- ☐ HTTPS, en-têtes de sécurité, secrets protégés.
- ☐ Sauvegardes et procédure de restauration.
- ☐ Guide de démarrage pour les développeurs.

> **MOT DE LA FIN**
>
> La théorie (HTTP, REST, architectures) donne le vocabulaire ; FastAPI donne les moyens ; la sécurité et le déploiement donnent la crédibilité. C'est la combinaison des trois qui transforme un exercice en API utilisable en production.